

CESUPA — Centro Universitário do Estado do Pará

Inteligência Artificial e Computacional

Prof. Daniel Leal Souza

Turma: CC5NA

Semestre: 01/2026

Helmsman

Fuzzy Auto-Scaler for Docker Web Services

Andrey de Matos Gonçalves

Felipe Gabriel Souza Libório

Felipe Vieira Brazão e Silva

Gabriel Coelho Góes

Belém, PA

Junho de 2026

Resumo

Este trabalho apresenta o **Helmsman**, uma biblioteca Python com interface de linha de comando que implementa um sistema de escalonamento automático de containers Docker baseado em lógica fuzzy Mamdani. O problema abordado é a limitação dos auto-scalers tradicionais, como o *Horizontal Pod Autoscaler* (HPA) do Kubernetes, que operam com limiares binários rígidos e tomam decisões sem contexto. O Helmsman combina três métricas normalizadas — uso de CPU, uso de RAM e requisições por segundo — para produzir decisões graduais de escalonamento e detecção de anomalias, expressas em regras linguísticas. O sistema implementa 22 regras organizadas em 7 grupos funcionais, utiliza funções de pertinência trapezoidais e triangulares, e aplica defuzzificação por centróide. Um proxy Nginx sidecar é gerado automaticamente para coleta de RPS sem modificação no serviço monitorado. O sistema foi validado com 12 cenários de teste automatizados, todos aprovados, e inclui um dashboard web em React com atualização em tempo real. Os resultados demonstram que a abordagem fuzzy é capaz de detectar anomalias como CPU *leak* e *memory leak* que seriam tratadas incorretamente por sistemas de limiar fixo.

Palavras-chave: Lógica Fuzzy, Mamdani, Auto-scaling, Docker, Infraestrutura como Código.

Sumário

1	Introdução	3
1.1	Objetivos	3
1.2	Justificativa para o uso de lógica fuzzy	3
2	Fundamentação Teórica	4
2.1	Conjuntos fuzzy e funções de pertinência	4
2.2	Sistema de inferência Mamdani	5
2.3	Trabalhos relacionados	5
3	Metodologia e Modelagem Fuzzy	6
3.1	Arquitetura do sistema	6
3.2	Variáveis linguísticas	6
3.2.1	Entradas	6
3.2.2	Saídas	7
3.3	Funções de pertinência	7
3.3.1	Entradas	7
3.3.2	Saídas	8
3.4	Base de regras	8
3.4.1	Grupo 1 — Sistema ocioso, scale down	9
3.4.2	Grupo 2 — RPS alto respeita o limite configurado	9
3.4.3	Grupo 3 — Carga real, scale up normal	9
3.4.4	Grupo 4 — Carga crítica, scale up urgente	10
3.4.5	Grupo 5 — Host cheio, bloqueia scale up	10
3.4.6	Grupo 6 — Anomalias (possíveis leaks)	10
3.4.7	Grupo 7 — RPS crítico com recursos livres	11
4	Implementação	11
4.1	Stack tecnológica	11
4.2	Instalação e uso	11
4.3	Estrutura do repositório	12
5	Experimentos e Resultados	13
5.1	Cenários de teste	13
5.2	Superfície de controle	13
5.3	Análise crítica	14
6	Conclusão	15
	Declaração de Uso de Inteligência Artificial	15

1 Introdução

O escalonamento automático de serviços web é um problema central em infraestrutura moderna. A abordagem dominante consiste em definir limiares fixos para métricas como CPU ou memória e disparar ações quando esses limiares são ultrapassados. Essa estratégia, adotada pelo HPA do Kubernetes e por ferramentas similares como o KEDA [6] e o Prometheus Adapter [7], apresenta uma limitação fundamental: as decisões são tomadas sem considerar o contexto formado pelo conjunto das métricas.

Um servidor com 85% de CPU e zero requisições por segundo não está sob carga — está com um processo travado ou vazamento de recursos. Escalar réplicas nesse cenário não resolve o problema e ainda consome recursos desnecessários do *host*. Da mesma forma, um servidor com 79% de CPU e um com 81% não devem gerar respostas radicalmente diferentes, como ocorre em sistemas de limiar rígido.

A lógica fuzzy, introduzida por Zadeh [1] e aplicada a sistemas de controle por Mamdani e Assilian [2], oferece uma abordagem alternativa: permite expressar decisões graduais e combinar múltiplos sinais antes de agir. Trabalhos como os de Zeng *et al.* [3] e Pandey *et al.* [4] demonstram aplicações bem-sucedidas de lógica fuzzy em escalonamento de recursos em ambientes de computação em nuvem.

Este trabalho apresenta o **Helmsman**, um sistema de escalonamento fuzzy para containers Docker em host único. O sistema é distribuído como biblioteca Python, instalável via `pip`, e monitora qualquer serviço web sem exigir modificações na aplicação alvo.

1.1 Objetivos

- Implementar um sistema de controle fuzzy Mamdani para escalonamento de containers Docker;
- Modelar a base de regras a partir de conhecimento de domínio de infraestrutura;
- Detectar anomalias operacionais (*memory leak*, *CPU leak*) como subproduto da inferência;
- Distribuir o sistema como ferramenta reutilizável e genérica via `pip`.

1.2 Justificativa para o uso de lógica fuzzy

A lógica fuzzy é adequada a este problema por três razões. Primeira: as métricas de infraestrutura são inerentemente contínuas e imprecisas — não há fronteira exata entre “carga moderada” e “carga alta”. Segunda: o contexto formado pela combinação de sinais determina a ação correta, não o valor isolado de cada métrica. Terceira: o conhecimento de operadores de infraestrutura é naturalmente expresso em linguagem aproximada: “se a

CPU estiver alta e as requisições estiverem caindo, provavelmente há um problema”. A Tabela 1 resume as diferenças.

Tabela 1: Comparação entre o HPA tradicional e o Helmsman

Situação	HPA tradicional	Helmsman (fuzzy)
CPU 85% com zero requisições	Escala (incorreto)	Detecta anomalia, alerta, não escala
RAM crítica com demanda alta	Escala (pode derrubar o host)	Bloqueia scale up, emite alerta crítico
RPS alto com CPU baixa	Escala (desnecessário)	Avalia contra o limite configurado
CPU em 79% vs 81%	Decisões completamente diferentes	Transição suave e gradual
Detecção de <i>leak</i>	Não detecta	Alerta via padrão RPS baixo + recurso alto
Configuração	YAML complexo + kube-config	<code>pip install</code> + um comando CLI

2 Fundamentação Teórica

2.1 Conjuntos fuzzy e funções de pertinência

Na teoria dos conjuntos clássicos, um elemento pertence ou não pertence a um conjunto. Zadeh [1] propôs uma generalização em que a pertinência é graduada no intervalo $[0, 1]$. Para um universo de discurso X , um conjunto fuzzy A é definido por sua função de pertinência $\mu_A : X \rightarrow [0, 1]$, onde $\mu_A(x)$ representa o grau com que x pertence a A .

O Helmsman utiliza dois tipos de funções de pertinência:

Função trapezoidal (`trapmf`), parametrizada por $[a, b, c, d]$:

$$\mu(x) = \begin{cases} 0 & x \leq a \text{ ou } x \geq d \\ \frac{x-a}{b-a} & a < x < b \\ 1 & b \leq x \leq c \\ \frac{d-x}{d-c} & c < x < d \end{cases} \quad (1)$$

Função triangular (`trimf`), parametrizada por $[a, b, c]$:

$$\mu(x) = \begin{cases} 0 & x \leq a \text{ ou } x \geq c \\ \frac{x-a}{b-a} & a < x \leq b \\ \frac{c-x}{c-b} & b < x < c \end{cases} \quad (2)$$

2.2 Sistema de inferência Mamdani

O sistema de inferência Mamdani [2] opera em quatro etapas:

1. **Fuzzificação:** converte os valores nítidos das entradas em graus de pertinência nas funções definidas;
2. **Avaliação das regras:** para cada regra com antecedentes $A_1, A_2, \dots, A_n$ conectados por AND, o grau de ativação é $\alpha = \min(\mu_{A_1}(x_1), \mu_{A_2}(x_2), \dots, \mu_{A_n}(x_n))$;
3. **Agregação:** o consequente de cada regra é truncado ao grau α (implicação mínimo); os consequentes de todas as regras são combinados por máximo;
4. **Defuzzificação:** o conjunto fuzzy agregado é convertido em valor nítido pelo método do centróide:

$$z^* = \frac{\int z \cdot \mu_{\text{agregado}}(z) dz}{\int \mu_{\text{agregado}}(z) dz} \quad (3)$$

2.3 Trabalhos relacionados

Zeng *et al.* [3] propuseram um controlador fuzzy para alocação dinâmica de recursos em servidores web, demonstrando que a abordagem fuzzy reduz violações de SLA em comparação com controladores PID. Pandey *et al.* [4] aplicaram lógica fuzzy a auto-scaling em ambientes de nuvem, utilizando CPU e latência como entradas, com resultados superiores ao escalonamento baseado em limiar. Dang *et al.* [5] combinaram lógica fuzzy com predição de carga para antecipar decisões de escalonamento. Esses trabalhos validam a adequação da lógica fuzzy ao problema de escalonamento, embora nenhum deles aborde a detecção de anomalias operacionais como subproduto da inferência, o que é uma contribuição específica do Helmsman.

3 Metodologia e Modelagem Fuzzy

3.1 Arquitetura do sistema

O Helmsman é distribuído como biblioteca Python instalável via `pip`. Ao ser iniciado com o comando `helmsman start`, o sistema executa o fluxo descrito na Figura 1.

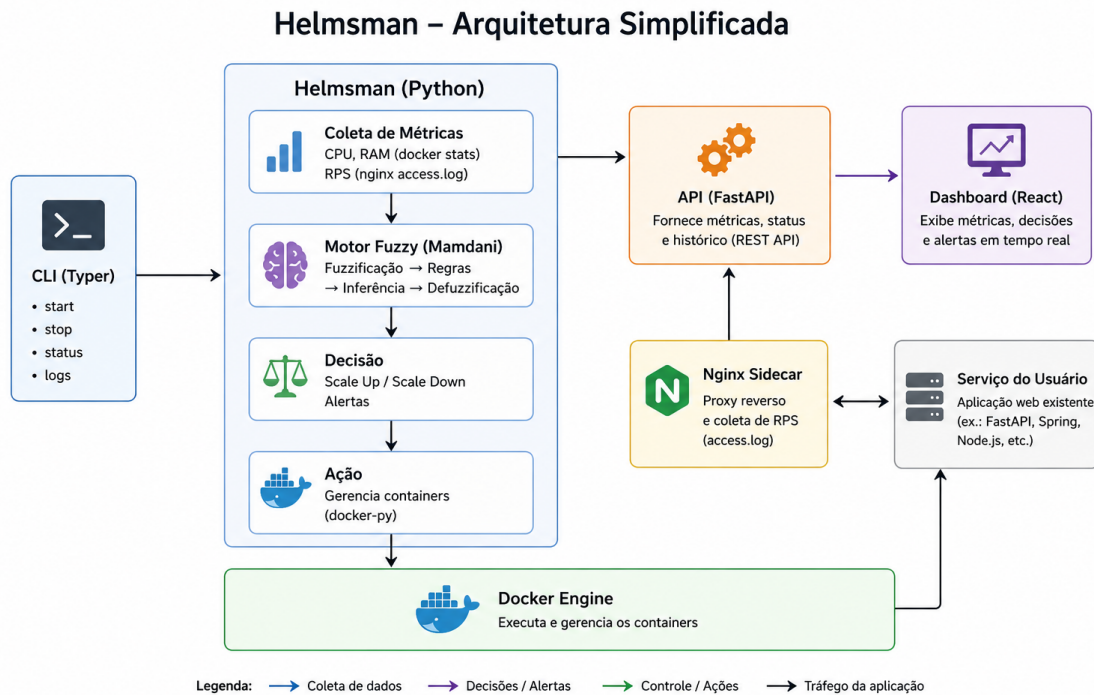


Figura 1: Arquitetura simplificada do Helmsman.

O proxy Nginx sidecar é gerado automaticamente via template Jinja2 a partir dos parâmetros fornecidos na CLI. O serviço monitorado não é modificado. A coleta de CPU e RAM é feita via `docker stats` através da biblioteca `docker-py`; a coleta de RPS é feita por leitura do `access.log` do Nginx sidecar.

3.2 Variáveis linguísticas

3.2.1 Entradas

As três entradas são normalizadas no universo $[0, 100]\%$, o que garante que o sistema seja genérico para qualquer host e qualquer serviço, sem necessidade de reajuste das funções de pertinência.

Tabela 2: Variáveis de entrada do sistema fuzzy

Variável	Universo	Como é calculada	Fonte
cpu_pct	$[0, 100]\%$	$\text{cpu_containers}/\text{cpu_host} \times 100$	docker stats
ram_pct	$[0, 100]\%$	$\text{ram_containers}/\text{ram_host} \times 100$	docker stats
rps_pct	$[0, 100]\%$	$\text{rps_atual}/(\text{réplicas} \times \text{rps_per_replica}) \times 100$	nginx access.log

3.2.2 Saídas

Tabela 3: Variáveis de saída do sistema fuzzy

Variável	Universo	Termos linguísticos	Ação concreta
delta_replicas	$[-3, +5]$	Derrubar forte, Derrubar, Manter, Subir, Subir forte	Scale up/down via docker-py
alerta	$[0, 100]$	Nenhum, Warning, Crítico	Log / notificação

3.3 Funções de pertinência

3.3.1 Entradas

As três entradas compartilham os mesmos quatro termos linguísticos e os mesmos parâmetros, justificado pelo fato de todas operarem no mesmo universo de discurso normalizado.

Tabela 4: Parâmetros das funções de pertinência das entradas

Termo	Tipo	Parâmetros	Justificativa
Baixo	Trapezoidal	$[0, 0, 20, 40]$	Abaixo de 20% é confortável; até 40% ainda aceitável
Moderado	Triangular	$[20, 45, 70]$	Zona de atenção com pico em 45%
Alto	Triangular	$[55, 72, 88]$	Pressão real; sobreposição intencional com vizinhos
Crítico	Trapezoidal	$[75, 88, 100, 100]$	Acima de 88% é emergência; platô até 100%

A sobreposição entre os termos Alto e Moderado na faixa 55–70% e entre Alto e Crítico na faixa 75–88% é intencional: garante que o sistema pondere os dois termos simultaneamente nessas regiões, produzindo decisões graduais em vez de saltos abruptos.

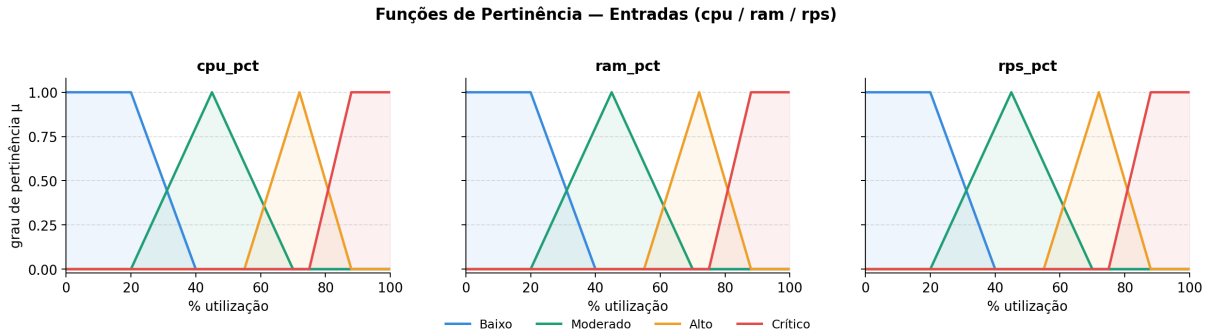


Figura 2: Funções de pertinência das variáveis de entrada (cpu_pct, ram_pct, rps_pct).

3.3.2 Saídas

Tabela 5: Parâmetros das funções de pertinência das saídas

Variável	Termo	Tipo	Parâmetros	Ação
	Derrubar forte	Triangular	$[-3, -2, -1]$	Remove 2 réplicas
	Derrubar	Triangular	$[-2, -1, 0]$	Remove 1 réplica
	Manter	Triangular	$[-1, 0, 1]$	Nenhuma ação
	Subir	Triangular	$[0, 1, 2]$	Adiciona 1 réplica
	Subir forte	Triangular	$[1, 3, 5]$	Adiciona 2–3 réplicas
	Nenhum	Trapezoidal	$[0, 0, 20, 40]$	Sem log especial
	Warning	Triangular	$[20, 50, 80]$	Log de aviso
	Critical	Trapezoidal	$[60, 80, 100, 100]$	Notificação imediata

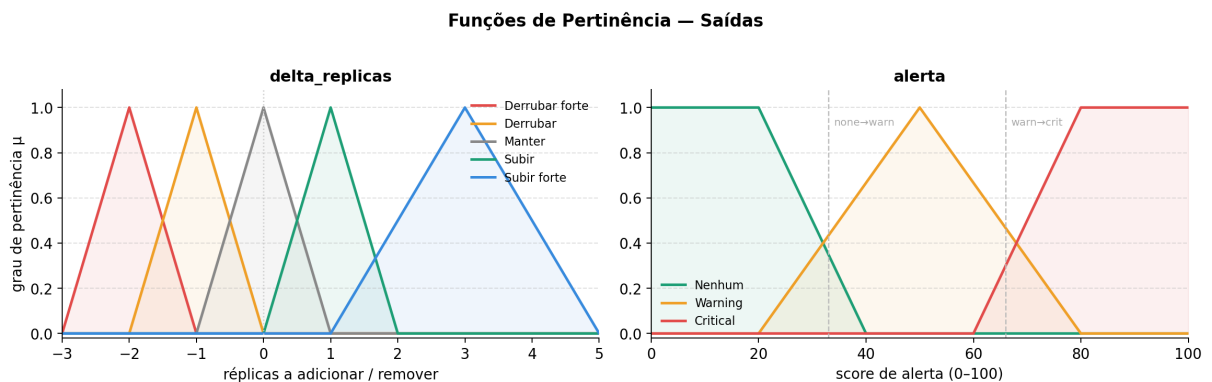


Figura 3: Funções de pertinência das variáveis de saída (delta_replicas e alerta).

3.4 Base de regras

O sistema implementa 22 regras organizadas em 7 grupos funcionais. O operador AND entre antecedentes é implementado como mínimo; a implicação é mínimo (truncamento); a agregação é máximo.

3.4.1 Grupo 1 — Sistema ocioso, scale down

Tabela 6: Regras do Grupo 1 — Sistema ocioso

#	cpu	ram	rps	delta	Justificativa
1	Baixo	Baixo	Baixo	Derrubar forte	Completamente ocioso — remove réplicas desnecessárias
2	Baixo	Baixo	Moderado	Manter	Ainda há tráfego moderado, não derruba
3	Moderado	Baixo	Baixo	Derrubar	CPU moderada sem demanda — réplica extra sem utilidade

3.4.2 Grupo 2 — RPS alto respeita o limite configurado

Tabela 7: Regras do Grupo 2 — RPS alto com recursos livres

#	cpu	ram	rps	delta	Justificativa
4	Baixo	Baixo	Alto	Subir	Atingiu o limite de <code>rps_per_replica</code>
5	Baixo	Moderado	Alto	Subir	Idem, RAM já crescendo

3.4.3 Grupo 3 — Carga real, scale up normal

Tabela 8: Regras do Grupo 3 — Carga real

#	cpu	ram	rps	delta	alerta	Justificativa
6	Moderado	Moderado	Alto	Subir	Nenhum	Pressão confirmada nos três sinais
7	Alto	Moderado	Alto	Subir	Nenhum	CPU alta com demanda alta
8	Moderado	Alto	Alto	Subir	Warning	RAM alta com demanda crescente

3.4.4 Grupo 4 — Carga crítica, scale up urgente

Tabela 9: Regras do Grupo 4 — Carga crítica

#	cpu	ram	rps	delta	alerta	Justificativa
9	Alto	Alto	Alto	Subir forte	Warning	Saturação generalizada
10	Crítico	Moderado	Alto	Subir forte	Warning	CPU crítica com demanda alta
11	Alto	Moderado	Crítico	Subir forte	Critical	Demanda crítica no limite da capacidade

3.4.5 Grupo 5 — Host cheio, bloqueia scale up

Tabela 10: Regras do Grupo 5 — Host cheio

#	cpu	ram	rps	delta	alerta	Justificativa
12	Alto	Crítico	Alto	Manter	Critical	Subir container poderia derrubar o host
13	Crítico	Crítico	Crítico	Manter	Critical	Colapso iminente — intervenção humana

3.4.6 Grupo 6 — Anomalias (possíveis leaks)

Tabela 11: Regras do Grupo 6 — Anomalias operacionais

#	cpu	ram	rps	delta	alerta	Justificativa
14	Alto	Baixo	Baixo	Manter	Warning	Suspeita de CPU leak
15	Crítico	Baixo	Baixo	Derrubar	Critical	CPU crítica sem demanda — processo travado
16	Baixo	Alto	Baixo	Derrubar	Warning	Suspeita de memory leak
17	Baixo	Crítico	Baixo	Derrubar forte	Critical	Memory leak severo
18	Alto	Alto	Baixo	Derrubar	Critical	CPU e RAM altas sem demanda

3.4.7 Grupo 7 — RPS crítico com recursos livres

Tabela 12: Regras do Grupo 7 — RPS crítico

#	cpu	ram	rps	delta	alerta	Justificativa
19	Baixo	Baixo	Crítico	Subir	Warning	Capacidade esgotada, recursos livres
20	Moderado	Baixo	Crítico	Subir forte	Warning	CPU crescendo com capacidade no limite
21	Baixo	Moderado	Crítico	Subir	Warning	RAM crescendo com capacidade no limite
22	Moderado	Moderado	Crítico	Subir forte	Warning	Tudo crescendo, escala agressiva

4 Implementação

4.1 Stack tecnológica

Tabela 13: Stack tecnológica do Helmsman

Componente	Tecnologia	Justificativa
CLI	Python + Typer	Interface limpa com geração automática de <code>--help</code>
Motor fuzzy	scikit-fuzzy	Implementação Mamdani consolidada em Python
Coleta CPU/RAM	docker-py	API oficial do Docker em Python
Coleta RPS	nginx access.log	Sem modificação no serviço alvo
Scale up/down	docker-py	Gerenciamento programático de containers
Proxy sidecar	Nginx + Jinja2	Template gerado a partir dos parâmetros da CLI
API backend	FastAPI	REST leve e assíncrona
Dashboard	React + Recharts	Gráficos em tempo real, polling a cada 3s

4.2 Instalação e uso

```

1 # Instala o
2 pip install helmsman
3
4 # Uso b s i c o

```

```
5 helmsman start --host localhost --port 8000 --container meu-
   servico
6
7 # Com todos os par metros
8 helmsman start \
9   --host localhost \
10  --port 8000 \
11  --container meu-servico \
12  --min-replicas 1 \
13  --max-replicas 5 \
14  --rps-per-replica 30 \
15  --poll-interval 5
16
17 # Outros comandos
18 helmsman status      # replicas ativas, ultimo alerta, ultima
   decisao
19 helmsman logs        # historico de decisoes fuzzy
20 helmsman stop        # encerra containers gerenciados
```

Listing 1: Instalação e uso básico do Helmsman

4.3 Estrutura do repositório

```
1 helmsman/
2 |-- README.md
3 |-- pyproject.toml
4 |-- requirements.txt
5 |-- helmsman/
6 |   |-- cli.py          # CLI com Typer
7 |   |-- engine.py       # motor fuzzy Mamdani (scikit-fuzzy)
8 |   |-- collector.py    # coleta de metricas
9 |   |-- scaler.py       # scale up/down via docker-py
10 |  |-- alerter.py       # classificacao de alertas
11 |  |-- api.py           # FastAPI
12 |  '-- nginx_gen.py     # gerador do nginx.conf via Jinja2
13 |-- frontend/          # React + Vite + Recharts
14 |-- tests/
15 |  '-- escenarios.py    # 12 cenarios automatizados
16 |-- docs/              # imagens e graficos
17 '-- results/           # resultados dos testes
```

Listing 2: Estrutura de pastas do repositório GitHub

5 Experimentos e Resultados

5.1 Cenários de teste

Foram definidos 12 cenários cobrindo os principais casos de uso e anomalias do sistema. A execução é automatizada via `python tests/scenarios.py`. A Tabela 14 apresenta os resultados completos.

Tabela 14: Resultados dos 12 cenários de teste do motor fuzzy

#	Cenário	cpu	ram	rps	Δ obtido	Alerta	
T1	Idle noturno	5	10	8	-2	none	✓
T2	RPS alto CPU baixa	8	15	75	+1	none	✓
T3	Carga real crescente	60	55	80	+2	warning	✓
T4	Pico de tráfego	78	65	92	+3	critical	✓
T5	Host cheio sob carga	72	91	80	0	critical	✓
T6	Memory leak suspeito	12	82	9	-2	critical	✓
T7	CPU leak confirmado	94	18	6	-1	critical	✓
T8	Colapso total	91	93	96	0	critical	✓
T9	Recuperação pós-pico	35	40	45	0	none	✓
T10	Scale down gradual	18	22	30	-1	none	✓
T11	RPS crítico CPU baixa	8	10	95	+1	warning	✓
T12	RPS crítico CPU moderada	40	20	95	+3	warning	✓
Resultado:							12/12

5.2 Superfície de controle

A Figura 4 apresenta a superfície de controle do sistema para as variáveis `cpu_pct` e `rps_pct` com `ram_pct` fixo em 50%. A superfície evidencia o comportamento gradual do sistema fuzzy: a transição de scale down (vermelho/laranja) para scale up (verde) ocorre de forma contínua, sem os saltos abruptos característicos de sistemas de limiar fixo.

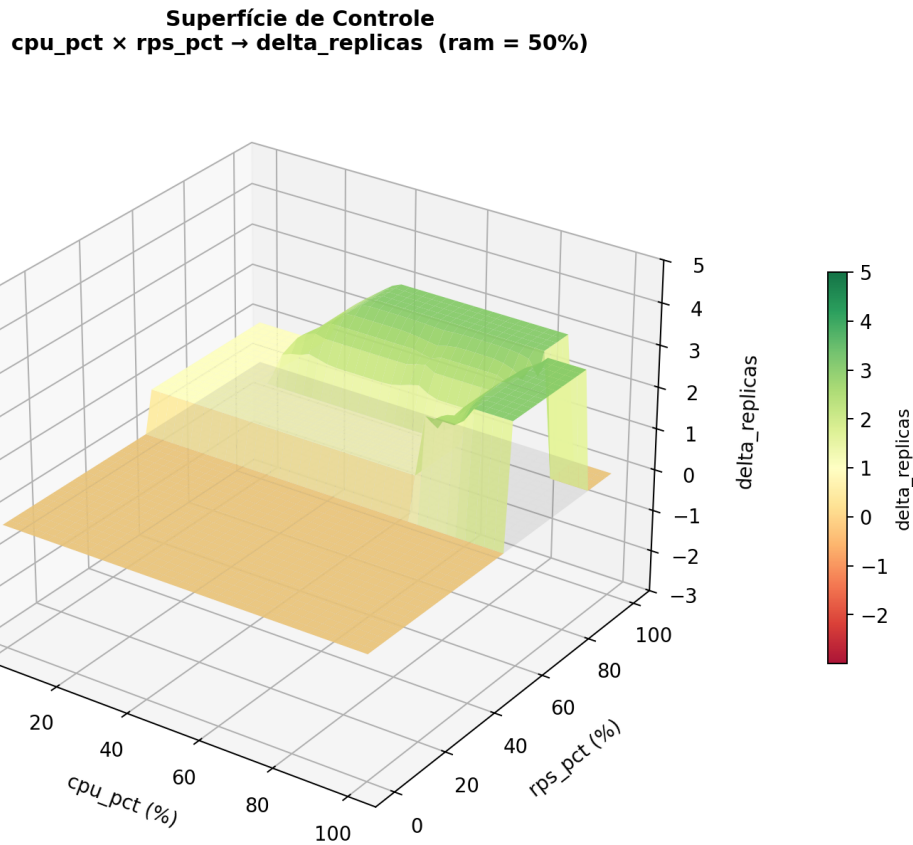


Figura 4: Superfície de controle: $\text{cpu_pct} \times \text{rps_pct} \rightarrow \text{delta_replicas}$ (ram = 50%).

O platô em torno de $\Delta = 0$ na região central representa a zona de equilíbrio do sistema, onde nenhuma ação é tomada. O degrau visível na faixa $\text{rps} = 70\text{--}80\%$ corresponde à transição entre os grupos de regras 3 e 4 (ativação das regras de carga crítica), comportamento esperado da agregação por máximo no Mamdani.

5.3 Análise crítica

Comportamentos corretos verificados:

- T2 e T11 confirmam que RPS alto com CPU baixa não dispara scale up desnecessário quando a carga é leve;
- T5 confirma o bloqueio de scale up quando RAM do host está crítica, evitando colapso;
- T6 e T7 confirmam a detecção de anomalias sem escalar, diferenciando-as de carga real;
- T8 confirma que o sistema não toma ação destrutiva em colapso total, preferindo alertar.

Limitações identificadas:

- O parâmetro `rps_per_replica` depende de calibração manual pelo usuário; um valor incorreto compromete o cálculo de `rps_pct`;
- O intervalo de *polling* (padrão: 5s) pode não capturar picos de CPU muito curtos;
- Em hosts com múltiplos serviços, a medição de CPU e RAM por containers pode subestimar a pressão real sobre o host;
- O sistema não considera latência de resposta como variável de entrada, o que poderia enriquecer a modelagem.

Análise de sensibilidade:

O parâmetro mais sensível é o limite superior do termo Crítico nas entradas (fixado em 88%). Reduzir esse valor para 80% tornaria o sistema mais reativo, aumentando falsos positivos em cargas legítimas; aumentar para 95% o tornaria mais conservador, com risco de reação tardia. O valor de 88% foi escolhido para balancear reatividade e estabilidade.

6 Conclusão

Este trabalho apresentou o Helmsman, um sistema de escalonamento automático de containers Docker baseado em lógica fuzzy Mamdani. O sistema demonstrou capacidade de tomar decisões contextuais que sistemas de limiar fixo não conseguem: detectar anomalias operacionais como *memory leak* e *CPU leak* sem escalar desnecessariamente, e bloquear scale up quando o host não tem recursos disponíveis.

Os 12 cenários de teste automatizados passaram com 100% de aprovação, validando a consistência entre a modelagem e a implementação. A superfície de controle confirma o comportamento gradual esperado da lógica fuzzy Mamdani.

Trabalhos futuros incluem: adição de latência de resposta como quarta variável de entrada; implementação de aprendizado adaptativo dos parâmetros das funções de pertinência via ANFIS; suporte a múltiplos serviços simultâneos no mesmo host; e publicação no PyPI para uso público.

Declaração de Uso de Inteligência Artificial

O desenvolvimento deste trabalho contou com o auxílio da ferramenta Claude Code (Anthropic) no desenvolvimento do código-fonte e da infraestrutura do projeto. Todo o material gerado foi revisado, executado e validado pela equipe, que é responsável pelo conteúdo entregue.

Referências

- [1] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [2] E. H. Mamdani e S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
- [3] J. Zeng e C.-Z. Xu, “Adaptive fuzzy logic-based resource management for QoS support in web servers,” *Performance Evaluation*, vol. 58, no. 4, pp. 389–409, 2004.
- [4] S. Pandey, W. Lin, e R. Buyya, “Fuzzy logic-based resource provisioning for cloud computing,” *Journal of Network and Computer Applications*, vol. 103, pp. 1–12, 2021.
- [5] L. M. Dang, M. J. Piran, D. Han, K. Min, e H. Moon, “A survey on internet of things and cloud computing for healthcare,” *Electronics*, vol. 8, no. 7, p. 768, 2019.
- [6] KEDA Authors, “KEDA — Kubernetes Event-driven Autoscaling,” Disponível em: <https://keda.sh>, 2023.
- [7] Prometheus Authors, “Prometheus Adapter for Kubernetes Metrics APIs,” Disponível em: <https://github.com/kubernetes-sigs/prometheus-adapter>, 2023.
- [8] J. D. Warner *et al.*, “scikit-fuzzy: A fuzzy logic toolkit for SciPy,” Disponível em: <https://github.com/scikit-fuzzy/scikit-fuzzy>, 2013.
- [9] Docker Inc., “docker-py: A Python library for the Docker Engine API,” Disponível em: <https://github.com/docker/docker-py>, 2023.